OptimaTech

AI Service — Integration & Deployment Guide

For LMS Core Backend & Frontend Teams

Version 1.0 | MVP | 21 July 2026

What's in this document

- Section 1 — Architecture overview: where the AI Service fits
- Section 2 — Local setup: run the AI Service with docker compose
- Section 3 — Integration contract: every endpoint the LMS team needs
- Section 4 — LMS Core responsibilities: what you must do before calling us
- Section 5 — Frontend integration: how to wire up the chat UI
- Section 6 — Environment variables reference
- Section 7 — Troubleshooting common errors

1. Architecture Overview

The AI Service is a standalone microservice. It runs independently from LMS Core and communicates through two simple integration points:

Integration Point	Direction	When
`POST /v1/ingest/video`	LMS Core → AI Service	After instructor uploads and finalizes a video
`POST /v1/rag/chat`	LMS Core → AI Service	When a student sends a chat message while watching a video

ℹ️ **NOTE:** The AI Service never calls the frontend directly. LMS Core acts as the proxy for all chat requests — it authenticates the user, then forwards the request to the AI Service.

2. Local Setup — Run with Docker Compose

The AI Service ships as a ready-to-run docker compose stack. You do not need Python, Node.js, or any other runtime installed — only Docker Desktop.

2.1 Prerequisites

- Docker Desktop installed and running
- Git installed
- Azure OpenAI credentials (API key + endpoint) — request from AI team
- The shared internal API key — request from AI team

2.2 Clone and configure

Step 1 — Clone the repository:

```
git clone https://github.com/your-org/optima-ai-service.git
cd optima-ai-service
```

Step 2 — Create your .env file:

```
cp .env.example .env
```

Step 3 — Fill in the required values in .env (see Section 6 for full reference):

```
INTERNAL_API_KEY=<key provided by AI team>
AZURE_OPENAI_API_KEY=<key provided by AI team>
AZURE_OPENAI_ENDPOINT=https://zeyad2047.openai.azure.com/
AZURE_OPENAI_API_VERSION=2024-12-01-preview
```

⚠️ **IMPORTANT:** Never commit the .env file to git. It is already in .gitignore.

2.3 Start the stack

```
docker compose up --build -d
```

This starts four containers:

Container	Port	Purpose
`api`	8000	FastAPI app — all HTTP endpoints
`worker`	—	Celery worker — runs video ingestion pipeline in background
`postgres`	5432	PostgreSQL 16 with pgvector extension
`redis`	6379	Redis — session memory + job queue

2.4 Verify it is running

```
curl http://localhost:8000/v1/health
```

Expected response:

```
{"status": "ok"}
```

Open the interactive API docs in your browser:

```
http://localhost:8000/docs
```

3. Integration Contract

All endpoints except /v1/health require the shared internal API key in a request header:

```
X-Internal-Api-Key: <key provided by AI team>
```

All request and response bodies are JSON. All IDs are UUID v4 strings. All time values are integers in milliseconds.

3.1 Health Check

Method	`GET /v1/health`
Auth	None required
Response	`200 {"status": "ok"}`
Use	Docker healthcheck, uptime monitoring, readiness probe

3.2 Ingest Video

Method	POST /v1/ingest/video
Auth	X-Internal-Api-Key header required
Returns	202 Accepted immediately — processing runs in background
409	Conflict — duplicate idempotency_key

Request body:

```
{
  "job_id":          "<uuid>",          // caller-generated
  "tenant_id":       "<uuid>",
  "course_id":       "<uuid>",
  "video_id":        "<uuid>",
  "video_title":     "Introduction to ML",
  "video_url":       "https://...",     // direct download URL, no auth required
  "language_hint":   "en",              // or 'ar' for Arabic
  "duration_ms":     3600000,
  "idempotency_key": "<video_id>:transcribe_index"
}
```

⚠️ **IMPORTANT:** video_url must be directly downloadable — no redirect to a login page. Use signed URLs (S3 presigned URL, Google Drive direct download, etc.).

3.3 Check Job Status

Method	GET /v1/jobs/{job_id}
Auth	X-Internal-Api-Key header required
Returns	200 with job status object

Status values and what they mean for the UI:

Status	UI Behavior
PENDING	Show 'Video is being queued for processing...'
PROCESSING	Show 'Video is being indexed, chat will be available soon...'
READY	**Enable chat UI — students can now ask questions**
READY_WITH_WARNINGS	Enable chat UI but transcript may have lower accuracy
FAILED	Show error state, allow instructor to retry via reindex endpoint

ℹ️ **NOTE:** Poll every 10 seconds. A typical 60-minute lecture takes 45-90 seconds to fully process.

3.4 RAG Chat

Method	POST /v1/rag/chat
Auth	X-Internal-Api-Key header required
Returns	200 with answer, citations, scope_used, grounded
422	VIDEO_NOT_READY — video not yet indexed

Request body:

```
{
  "request_id":   "<uuid>",    // unique per request, for tracing
  "tenant_id":    "<uuid>",
  "user_id":      "<uuid>",    // authenticated user ID from LMS session
  "course_id":    "<uuid>",
  "video_id":     "<uuid>",    // currently playing video
  "session_id":   "<uuid>",    // stable per chat session — same value for
                               //   all messages in one conversation
  "playback_ms":  75000,       // current video player position in ms
  "message":      "What is a residual?"
}
```

Response body:

```
{
  "request_id":           "<uuid>",
  "session_id":           "<uuid>",
  "answer":               "A residual is the difference between...",
  "scope_used":           "CURRENT_VIDEO",
  "grounded":             true,
  "citations": [
    {
      "video_id":     "<uuid>",
      "video_title":  "Linear Regression in 3 Minutes",
      "start_ms":     0,
      "end_ms":       145279,
      "label":        "00:00-02:25"
    }
  ],
  "follow_up_suggestions": []
}
```

ℹ️ **NOTE:** If grounded=false, the answer will be a polite refusal ('I can\'t find that in the current video.'). Always render the answer field regardless of grounded value.

3.5 Delete Video

Method	`DELETE /v1/videos/{video_id}`
Auth	X-Internal-Api-Key header required
Returns	204 No Content (idempotent)
Use	Call when instructor deletes or unpublishes a video in LMS

4. LMS Core Responsibilities

The AI Service trusts the caller. It never re-validates user auth or enrollment. The LMS Core backend is responsible for:

Responsibility	Detail
Authenticate user before chat	Verify JWT/session is valid before calling /v1/rag/chat
Verify enrollment	Confirm user is enrolled in course_id before forwarding request

Responsibility	Detail
Pass authoritative IDs	Never trust client-provided course_id or video_id — use server-side values
Trigger ingest on video upload	Call POST /v1/ingest/video after video upload is finalized
Track processing status	Poll GET /v1/jobs/{job_id} and store status in LMS DB to show in UI
Generate stable session_id	Create one UUID per chat session, reuse it for all messages in that session
Trigger delete on video removal	Call DELETE /v1/videos/{video_id} when instructor removes a video

5. Frontend Integration

5.1 Video indexing state

After an instructor uploads a video, the LMS backend should poll the AI Service for job status and expose it to the frontend. Show these states in the UI:

- PENDING / PROCESSING → show a banner: 'AI chat is being set up for this video...'
- READY → enable the chat panel
- FAILED → show: 'AI chat is unavailable for this video' with a retry option for instructors

5.2 Chat UI flow

Standard chat UI — send message, receive response. Key integration points:

Field	How to use it
`answer`	Render as markdown — the LLM formats with bullet points and citations inline
`grounded`	If false, style the answer differently (e.g. muted color) to indicate it's a refusal
`citations`	Render as clickable chips below the answer. On click: seek the video player to start_ms using player.seekTo(start_ms / 1000)
`label`	Display this as the chip label e.g. [00:00-02:25]
`session_id`	Generate once when the student opens the chat panel for a video. Reuse for all messages in that session. Reset when they navigate to a new video.
`playback_ms`	Pass the current video player position in milliseconds with every message. This improves answer relevance for time-sensitive questions.

5.3 Citation seek — JavaScript example

```
// On citation chip click:
function seekToTimestamp(startMs) {
  const seconds = startMs / 1000;
  videoPlayer.seekTo(seconds);  // adjust for your player API
}
```

5.4 Arabic / RTL support

The AI Service automatically detects non-English queries and responds in the student's language. No frontend changes are required for Arabic support — the answer field will contain Arabic text when the student asks in Arabic. Ensure your chat UI renders RTL text correctly for Arabic responses.

6. Environment Variables Reference

Variable	Required	Description
`INTERNAL_API_KEY`	**Yes**	Shared secret for service-to-service auth. Get from AI team.
`AZURE_OPENAI_API_KEY`	**Yes**	Azure OpenAI API key. Get from AI team.
`AZURE_OPENAI_ENDPOINT`	**Yes**	Azure OpenAI endpoint URL e.g. https://xxx.openai.azure.com/
`AZURE_OPENAI_API_VERSION`	**Yes**	API version e.g. 2024-12-01-preview
`DATABASE_URL`	Auto	Set automatically by docker-compose. Do not change.
`REDIS_URL`	Auto	Set automatically by docker-compose. Do not change.
`LMS_CALLBACK_BASE_URL`	Optional	LMS Core base URL for processing-status callbacks. Leave empty for MVP.

7. Troubleshooting

Error	Cause	Fix
`401 INVALID_API_KEY`	Missing or wrong X-Internal-Api-Key header	Get the correct key from the AI team and add it to every request
`409 DUPLICATE_JOB`	Same idempotency_key used twice	Expected behavior — the first job is already running or done
`422 VIDEO_NOT_READY`	Chat called before ingest finished	Poll GET /v1/jobs/{job_id} until READY before enabling chat
`Job status = FAILED`	Pipeline error (bad URL, STT failure, etc)	Check error_message in job status response. Fix the video URL and call POST /v1/ingest/video again with a new idempotency_key
`answer = empty string`	LLM token budget too low	Already fixed in current version. Update to latest code.
`grounded = false on valid question`	Video not yet indexed or wrong video_id	Confirm job status is READY and video_id matches the ingested video
`Container fails to start`	Missing .env values	Run: docker compose logs api — check for missing environment variable errors

Questions? Contact the AI Engineering team (Abdullah) before integration.